

Threads

- Represents a single flow of control; a sequence of executed instructions
- Multiple threads can run “concurrently”
 - True parallel
 - Interleaved
- Virtual machine starts a single thread

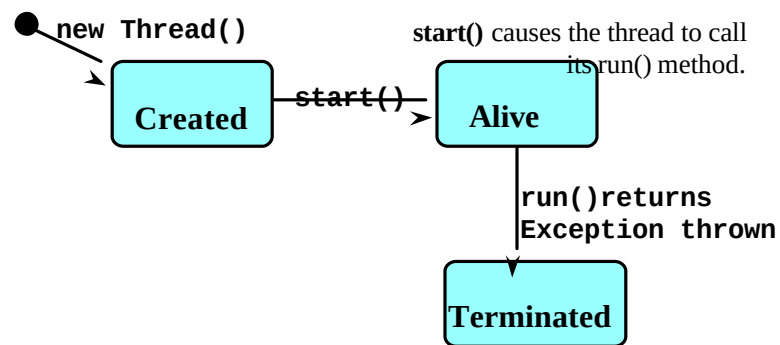
What You Need to Know About Threads

- What code does a thread execute?
- What states can a thread be in?
- How does a thread change its state?
- How does synchronization work?

Thread Objects

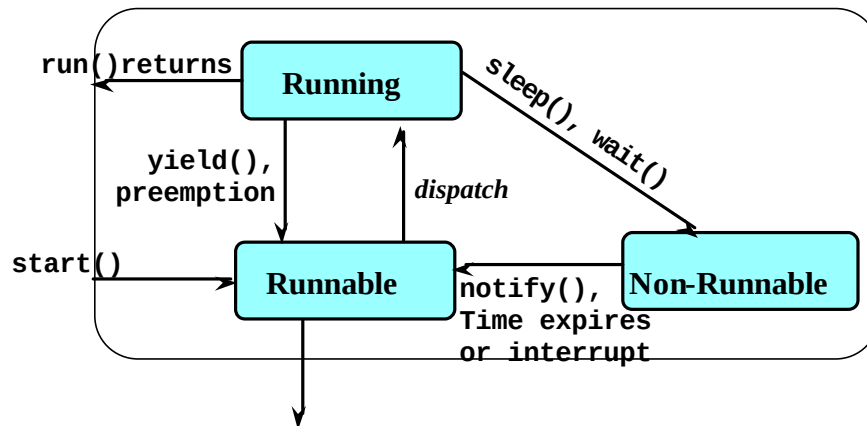
- Thread is an object
 - Must instantiate the thread before it executes
 - Usually sub-class `Thread` class
- Thread object can exist in several states
- `start()` method places thread in JVM's runnable queue
- When selected by scheduler `run()` method is executed.
- Thread terminates by
 - Exiting `run()`
 - Throwing an exception that is not caught

Thread Life-cycle in Java



Once terminated, it cannot be restarted.
The object state is available still.

Thread Alive States in Java



CS3 - Threads

JRV

5

Interface Runnable – Second Thread Creation Mechanism

- Class can implement **Runnable**
 - **Runnable** classes have a **run()** method
- Instantiate a **Runnable** object
- Instantiate a thread giving it the **Runnable** object as a constructor argument

CS3 - Threads

JRV

6

Thread Scheduling

- Scheduler decides which thread runs next
- Selection is among runnable threads
- Selection is round-robin except
 - Threads have priorities.
 - Higher priority threads have preference

Scheduling Implementations

- Scheduling is typically either:
 - non-preemptive
 - preemptive
- Most Java implementations use preemptive scheduling.
 - the type of scheduler will depend on the JVM that you use.
 - In a non-preemptive scheduler a thread leaves the running state only when it is ready to do so.

Thread Priorities

- Range: 1 (low) to 10 (highest)
- The default priority is 5
 - You can use the constants
 - Thread.MAX_PRIORITY
 - Thread.MIN_PRIORITY
 - Thread.NORM_PRIORITY
- Priorities can be changed and accessed

isAlive()

- Determines if a thread is alive
 - Started but not dead
 - Runnable or blocked
- Can tell if thread terminated
- Can not distinguish between runnable and blocked

isAlive()

```
public class WorkerThread extends Thread {
    private int result = 0;

    public void run() {
        // Perform a complicated time consuming calculation
        // and store the answer in the variable result
    }

    public static void main(String args[]) {
        WorkerThread t = new WorkerThread();
        t.start();

        // Why is this "wasteful"?
        while ( t.isAlive() );

        System.out.println( result );
    }
}
```

sleep()

- Current thread blocked for period of time
 - sleep(int milliseconds)
 - sleep(int millisecs, int nanosecs)
- Sleep can throw an InterruptedException

sleep()

```
public class WorkerThread extends Thread {
    private int result = 0;

    public void run() {
        // Perform a complicated time consuming calculation
        // and store the answer in the variable result
    }

    public static void main(String args[]) {
        WorkerThread t = new WorkerThread();
        t.start();

        while ( t.isAlive() )
            try {
                sleep( 100 );
            } catch ( InterruptedException ex ) {}

        System.out.println( result );
    }
}
```

CS3 - Threads

JRV

13

yield()

```
public class WorkerThread extends Thread {
    private int result = 0;

    public void run() {
        // Perform a complicated time consuming calculation
        // and store the answer in the variable result
    }

    public static void main(String args[]) {
        WorkerThread t = new WorkerThread();
        t.start();

        while ( t.isAlive() )
            yield()

        System.out.println( result );
    }
}
```

CS3 - Threads

JRV

14

Joining Threads

- You really want to do nothing until a thread terminates
- `join()` does exactly that
 - `join()`
 - `join(long timeout)`
 - `join(long timeout, int nanos)`

`join()`

```
public class WorkerThread extends Thread {
    private int result = 0;

    public void run() {
        // Perform a complicated time consuming calculation
        // and store the answer in the variable result
    }

    public static void main(String args[]) {
        WorkerThread t = new WorkerThread();
        t.start();

        try {
            t.join();
        } catch ( InterruptedException ex ) {}

        System.out.println( result );
    }
}
```


Synchronization

- A new issue for multi-threaded programs
- Multiple threads interact by sharing data
- Uncontrolled interaction is not good
- It has caused systems to crash, and people to be hurt (re. Therac accident)

Object Locks

- Every object has a *lock*
- Only one thread at a time can hold the lock
- Acquire a lock by entering a synchronized block

```
synchronized( someObject) {  
    // must obtain someObject's lock to enter this block.  
}
```

- Shorthand to synchronize an entire method

```
public synchronized someMethod() { ... }
```

Object Locks (cont.)

- A thread gives up a lock by:
 - leaving a block of synchronized code
 - calling `wait()` on the lock object
- Is the object locked or the block of code?

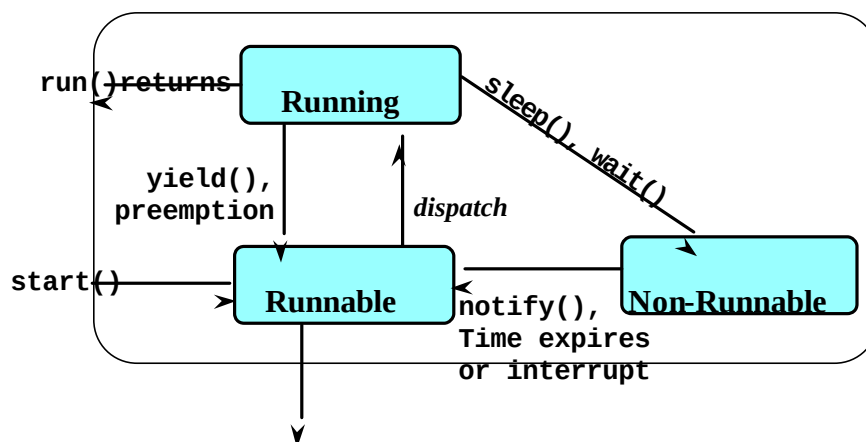
`wait()`

- A synchronized block forces a thread to wait if another thread is holding the object's lock
- A thread holding a lock can voluntarily be made non-runnable by executing `wait()` on the object
- Why voluntarily be made non-runnable?

notify()/notifyAll()

- A thread executing `wait()` can be released by:
 - `notify()` – release one waiting thread
 - `notifyAll()` – release all waiting threads
- Must hold lock to execute `notify()/notifyAll()`
- Why might this appear to be a problem?

Thread Alive States in Java



Class Lock

- Up to now each lock was associated with an object
- There is also a class lock
- To execute a synchronized `static` method the thread must obtain the class lock
- Only one thread can hold the class lock
 - Only one of each class lock per system
 - Every instantiated object has an object lock